ARCHITECTURE BLUEPRINT · V0.1

The Crate Lab

One manifest turns a published Rust workspace into a testable, UI-generatable surface. cratelab reads tv-rs and cctv-rs — typed command enums, capability sets, error taxonomies — and derives the rest: generative panels, scenario playlists, a coverage heatmap, and limit fuzzing against a mock fleet that never sleeps.

Zakaria Labib

tv-rs · cctv-rs · mediamaster-rs — crates as backend and facade

NOVEMBER 2026 · MOCK-FIRST, HARDWARE BEHIND A TOGGLE

Table of Contents

1. Why a lab 2

2. System architecture 3

3. The manifest: one artifact drives it all 4

4. Discovery and matching: local checkout or crates.io 5

5. One error vocabulary 6

6. What the lab tests: commands, events, servers, limits 7

7. Scenarios: playlists plus Rhai 8

8. Generative UI: the panel compiler 8

9. The coverage map 9

10. The scenario composer: cross-crate pipelines 10

11. Limit fuzzing and deterministic chaos 10

12. Expansion: the generative roadmap 11

13. Roadmap and getting started 11

1. Why a lab

Three workspaces are already published on crates.io. **tv-rs** is fourteen crates that find televisions on a LAN and drive them — SSDP discovery, Wake-on-LAN, Samsung Tizen, LG webOS, Roku ECP, MDC panels, DLNA, HDMI-CEC, a reconciling agent and an HLS media server behind one facade. **cctv-rs** is eleven crates that do the same for camera fleets: ONVIF, RTSP, RTP, Ezviz, UVC, detection zones and a deterministic reconciler. **mediamaster-rs** shares a hardware media vocabulary — event bus, audio zones, SDR, sensors. All three follow the same engineering religion: typed device models, capability sets that refuse honestly, zero panics, and errors as part of the API. What none of them has is a place to *sit* while you work on them.

cratelab is that place: a local bench with a webview, built in Rust with Tauri v2, that checks a workspace — online on crates.io or a local checkout — and makes it testable. The word *testable* is doing a lot of work, so let us sharpen it. It means every command, event, server and limit the crates expose becomes clickable from a UI that did not exist when the command was written. It means refusals, timeouts and rate limits can be exercised deliberately instead of being discovered by accident. It means a morning routine — wake the screens, verify power, launch content, reconcile the fleet — is a playlist you can run, diff and hand to CI. And it means all of that happens against a mock fleet by default, with real hardware one explicit toggle away.

The trick that makes this cheap is that the workspaces already declare themselves in Rust. tv-rs-core defines CommandId variants with doc comments; both device models carry capability sets, confidence-tagged state and an error taxonomy. The lab does not re-state any of it. It reads or generates a manifest of that surface, merges it with a few things only a human knows — which commands are destructive, what a sane timeout is, where the mock behaviour lives — and derives everything else from that one artifact.

7 crates
lab workspace, compiled + tested

34 tests
cargo test --workspace, offline

19 commands
declared in the two sample manifests

4 lab ideas
generative UI · coverage · composer · fuzzing

Four design rules carry over from the workspaces themselves, because a bench that betrays its own tools would be useless. Refusals are data: a capability refusal is a first-class outcome with severity *info*, not a red error. Nothing panics on any path a manifest or a script can influence. The lab is mock-first — the same port that serves the virtual fleet will later serve a real TvClient, so scenarios never know the difference. And failure injection is deterministic: counter-based chaos, no randomness, so a failing run replays identically until you change the policy.

2. System architecture

cratelab is a virtual Cargo workspace of seven small crates plus a Tauri v2 shell, kept deliberately boring in shape: one port, one ledger, one vocabulary. The shell is the only crate that touches pixels; the engine is the only crate that decides what happens next; the workspaces under test are behind a single trait, so the mock fleet and a future hardware adapter are interchangeable inside every scenario. The figure below shows the whole system — note how the manifests live *beside* the engine as data, not inside it as code.

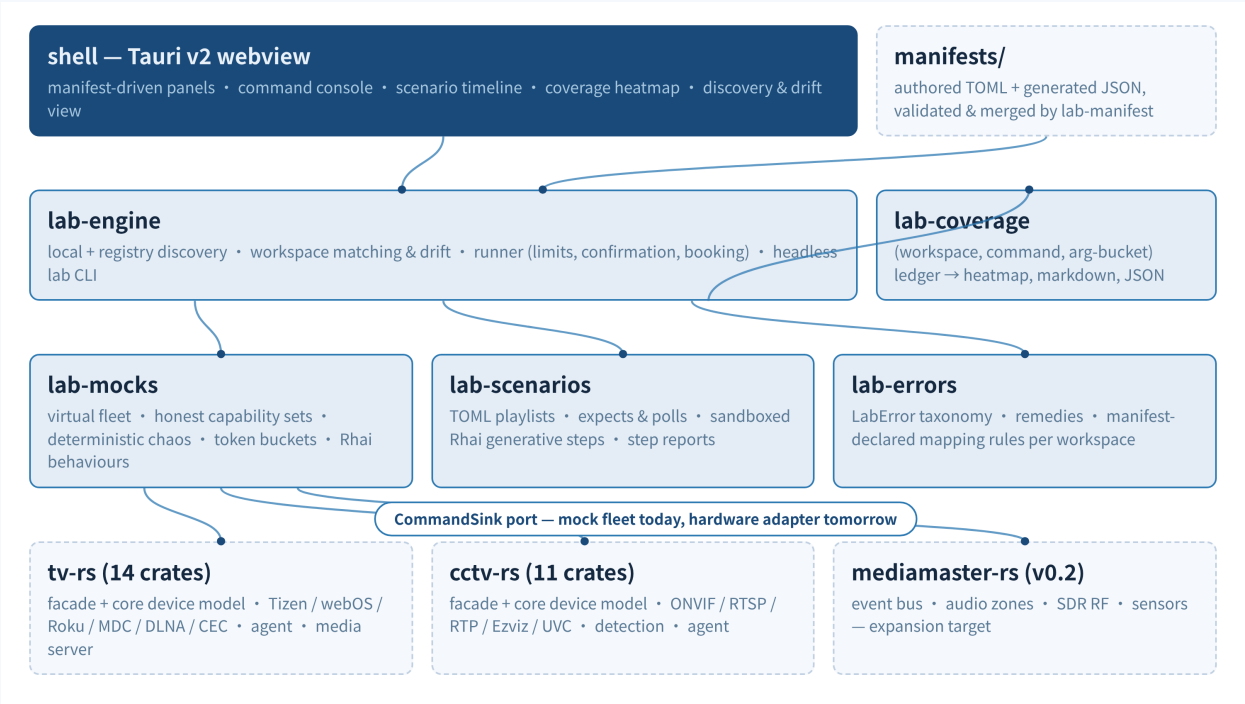


Figure 1 — cratelab architecture: adjacent layers only, the CommandSink port separates the lab from the workspaces.

Crate	Role	Key types
lab-errors	One error vocabulary for every workspace, with remedy hints and manifest-declared mapping rules	LabError, Remedy, ErrorMapping
lab-manifest	Schema for commands, events, servers, limits, device classes; validates and merges the two manifest layers	LabManifest, CommandSpec, GeneratedManifest
lab-generate	syn-based generator: typed CommandId enums become the generated JSON layer	generate_from_source()
lab-mocks	Virtual fleet, honest capability sets, deterministic chaos, token buckets, Rhai behaviours	MockFleet, VirtualDevice, ChaosPolicy
lab-scenarios	TOML playlists and sandboxed Rhai generative steps; step-level reports	Scenario, Step, ScenarioRunner
lab-coverage	The ledger: (workspace, command, arg-bucket) cells, heatmap rows, markdown and JSON reports	Ledger, CellKey, Outcome

Crate	Role	Key types
lab-engine	Discovery (local / registry), matching and drift, the runner, and the headless lab CLI	Lab, discovery, match_workspaces

Table 1 — the seven lab crates. The Tauri shell (shell/) is a standalone workspace on purpose: it needs the platform webview stack the core must not require.

Two details are worth calling out because they are load-bearing. First, the engine's invoke() path books coverage for *every* outcome before returning — including refusals and rate-limit rejections — so the heatmap is never blind to the interesting failures. Second, the shell keeps one persistent Lab in managed state, which means the command console, the scenario runner and the coverage tab all write into the same ledger: play with the UI for two minutes and the heatmap moves.

3. The manifest: one artifact drives it all

A lab manifest is a small TOML file per workspace — tv-rs.lab.toml, cctv-rs.lab.toml — that declares the surface in four nouns: commands, events, servers, limits. It is produced in two layers that meet in lab-manifest. The **generated layer** is emitted by lab-generate, which parses the workspace's real CommandId enum with syn: variants become stable dotted ids, doc comments become help text, and struct-variant fields become typed arguments. The **authored layer** is what a compiler cannot know: labels, danger levels, timeouts, rate limits, mock script bindings, server health routes, quirks. The generated layer proposes; the authored layer disposes.

Matching between the layers works on id *or* variant, so an authored command can keep a prettier dotted id (tv.power.off) while its generated counterpart is derived from the variant name (tv.power_off) — the merge pins them together through variant = "PowerOff". Anything generated but not authored still appears, as an attention-level skeleton, because the lab never hides a surface the workspace exposes. Validation refuses duplicate ids, unknown emits references, enum arguments without options, and ids that ignore the workspace prefix.

A slice of the tv-rs authored manifest:

```
[[commands]]
id = "tv.power.off"
label = "Power off"
kind = "action"
danger = "destructive"        # the UI asks; the runner refuses without --confirm
variant = "PowerOff"          # bridge to the generated layer
args = [
  { name = "ip", type = "ipv4" },
  { name = "vendor", type = "enum", options = ["samsung", "lg", "roku", "mdc"] },
]
timeout_ms = 8000
rate_limit = { per_second = 1, burst = 2 }
emits = ["tv.power.changed"]
```

The argument types are exactly the widget vocabulary of the generative UI: str, int, float, bool, enum, ipv4, mac, url, path, json.

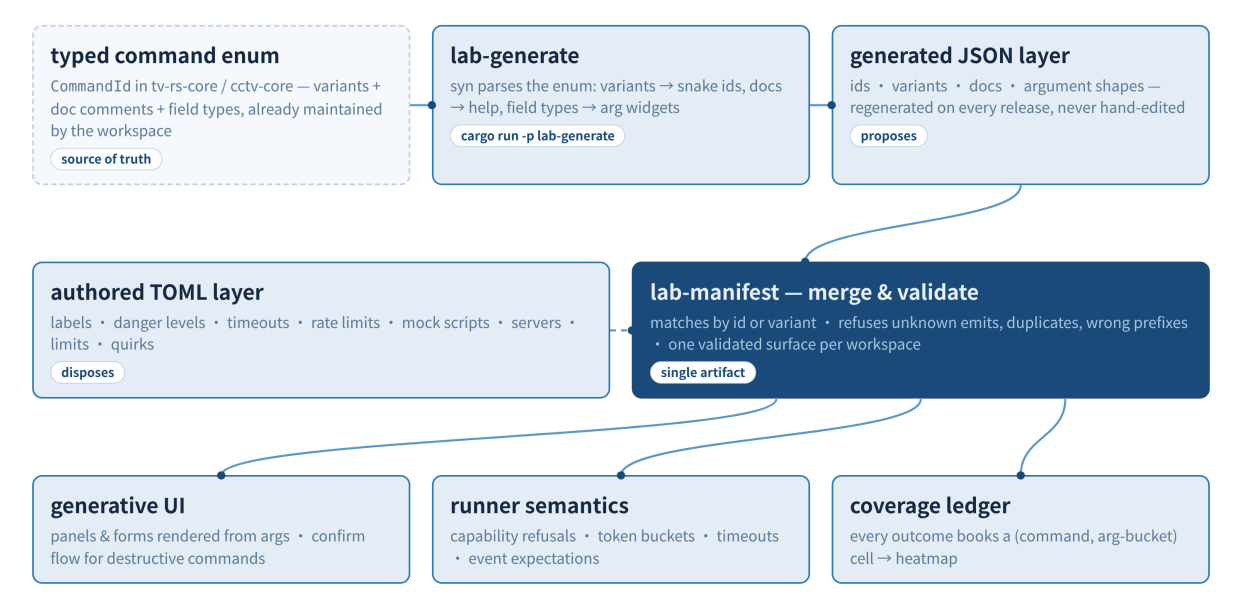


Figure 2 — the manifest lifecycle: the compiler's types feed the generator, the human disposes, one validated artifact drives UI, runner and ledger.

The generator is a real binary, not a sketch. Point it at the enum source and it writes manifests/generated/.commands.json plus a commented TOML skeleton for the authored layer. Because it runs in CI, a new variant that appears in a release shows up in the lab's next run even if nobody touched a manifest — the drift view in Chapter 4 is what tells you which checkout a manifest actually describes.

4. Discovery and matching: local checkout or crates.io

The lab has to answer a humble question before it can test anything: *what is actually here?* Local discovery reads a checkout's virtual Cargo.toml, resolves member globs such as crates/* against the filesystem, and reads each member's package name and version. It is deliberately dependency-light — no cargo metadata subprocess and no directory-walking crate — because discovery must work identically on a CI machine where cargo is not installed.

Registry discovery is the online twin. Behind the registry feature, lab-engine queries the crates.io *sparse index* rather than the REST API: plain JSON lines over HTTPS, no authentication, the same transport cargo itself uses, and it stays stable under load. Unknown crates degrade gracefully; yanked versions are skipped; the latest surviving version wins. The feature is off in the default test profile so cargo test --workspace runs fully offline.

Matching then reconciles three sources — the manifest, the local checkout, the registry — into one WorkspaceMatch with an honest drift verdict. The comparison is a three-component semver ordering implemented in twenty lines rather than pulled in as a dependency, which is enough to distinguish the only three states that matter to an operator.

Drift	Meaning	What the UI does
in_sync	Manifest version equals the tested version	Green chip; run freely
manifest_behind	The checkout or registry is newer than the manifest	Warn banner: surface may be stale, regenerate
manifest_ahead	The checkout is older than the manifest	Warn banner: results describe unreleased behaviour
unknown	No version information for comparison	Neutral chip; discovery decides later

Table 2 — drift vocabulary between manifests and reality.

The same discovery data drives the shell's Discovery tab: paste a workspace root, see the member crates with versions, and read the drift verdict next to each manifest. Because matching is pure data, the headless CLI can gate on it — a release script can refuse to publish when the lab manifest is behind the facade crate it claims to describe.

5. One error vocabulary

Each workspace already owns a typed error taxonomy; the lab must not replace it, it must *normalize* it. lab-errors defines eleven kinds — manifest, discovery, transport, capability, auth, timeout, rate-limited, scenario, script, coverage, internal — and every component in the lab branches on those and nothing else. Each LabError carries a **remedy**: a machine-readable suggestion of what should happen next, from a plain *retry* to *reauth*, *check-capabilities*, *fix-manifest* or *reduce-limits*. The UI renders the remedy instead of a stack trace; CI maps severity to exit codes (info = 0, warn = 2, fatal = 3).

Kind	Retryable?	Remedy	Typical origin
capability	no	check capabilities	device class lacks the command's capability
auth	no	reauth	pairing token rejected; LG client-key expired
timeout	yes	retry after delay	operation exceeded its manifest budget
rate_limited	yes	retry after N ms	token bucket refused the call
transport	declared	retry or abort	socket, TLS, WebSocket, RTSP failures
discovery	yes	retry	sweep failed, device vanished
manifest	no	fix manifest	validation refused the artifact
scenario	no	fix scenario	step assertion failed
script	no	fix script	Rhai misbehaved or returned garbage
internal	no	abort	a lab bug; never user-facing

Table 3 — the LabError kinds and their semantics. Capability refusals are severity info and exit code 0: expected during testing.

Workspaces are wired in through **manifest-declared mapping tables** instead of compile-time dependencies. A [[errors.mapping]] rule matches the source error's variant or kind string (exact or prefix) and maps it to a lab kind, with an optional retryability override and a note the UI can show. Unknown variants degrade to a retryable transport error, loudly, instead of silently vanishing. When the shell later drives a real TvClient, the adapter's only job is to hand the workspace error to its workspace's table; thirteen crates of tv-rs errors become eleven lab kinds without the lab knowing a single vendor type.

6. What the lab tests: commands, events, servers, limits

The runner's semantics follow directly from the manifest. A **command** is invoked through the CommandSink port: the fleet checks the device exists, the spec exists, the capability set admits it, the token bucket admits it, and chaos does not reject it — in that order, and the order matters because a capability refusal must never cost a socket. Only then does behaviour run: a Rhai script if the command declares one, otherwise the built-in responder that models the obvious verbs (wake turns power on, off turns it to standby, state reads it back).

Events are declared with their source — poll, GENA, WebSocket, MQTT or the in-process bus — and a payload type, mirroring how tv-rs really learns state: the agent polls, DLNA renders push GENA LastChange, Tizen and webOS speak over their WebSocket channels. A command that emits an event must find that event in the manifest, and mock scripts return their emissions in the reply, so the scenario runner can carry them forward as variables. **Servers** — tv-rs-media's range-capable HLS server, cctv-agent's snapshot proxy — are declared with crate, default port, health path and routes; the lab starts them, probes health, and treats them as peers of devices rather than configuration. **Limits** are first-class declarations (budget, rate, pool, concurrency) that name the component they govern — the budgeted WebSocket frame reader, the per-TV connection pool, the agent's one-action-per-hour invariant — and Chapter 11 turns them into targets.

Surface	Declared by	Exercised by	Observed through
Commands (19)	commands + args + danger	command console, playlists	SinkOutcome + coverage cells
Events (9)	events + source + payload	emits from commands, poll steps	timeline vars, bus
Servers (2)	servers + health route	lab-started, probed	health endpoint
Limits (8)	limits + kind + applies_to	rate limiter, chaos, fuzz	refusals as coverage

Table 4 — the four nouns of a lab manifest, counted across the two sample manifests.

7. Scenarios: playlists plus Rhai

A scenario is a timeline of steps in a TOML playlist. Declarative steps run a command with arguments, an expectation (*ok*, a specific error kind, or a device state that must hold), a per-step timeout and an optional poll policy for eventually-consistent devices — the agent's world, where power state arrives when it arrives. Variables flow forward: a scenario declares seed variables, steps can reference them as $vars.name, and any step may add new ones. When a step fails, on_fail decides between abort and continue, so a fixed-dome camera that refuses PTZ does not hide the detection results of the steps after it.

```
[[steps]]
run = "tv.power.state"
args = { ip = "$vars.ip", vendor = "$vars.vendor" }
expect_state = { key = "power", equals = "On" }
poll = { attempts = 5, interval_ms = 100 }

[[steps]]
run = "cctv.ptz.goto_preset"
device = "uvc-02"                 # a stream-only adapter
args = { ip = "$vars.cam", preset = 1 }
expect = { error_kind = "capability" }   # refusals are assertions too
```

Declarative steps from the two sample playlists. The second expects a refusal — the lab treats the workspace's refusal vocabulary as a testable contract.

Rhai steps are the generative half of the same playlist. They run in a hard sandbox — bounded operations, bounded string sizes, no I/O — under the contract fn step(vars_json) -> String returning lab-JSON: a verdict, an optional message, and new variables for later steps. That is enough to synthesize a flaky hour, pick the next vendor round-robin, or generate an argument matrix before the declarative steps consume it. Mock command behaviour uses the same language under a second contract (fn on_command(cmd_id, args)), so one mental model covers both ends of the wire.

8. Generative UI: the panel compiler

The shell's central promise is that the UI is *derived*, not hand-written. The commands view renders one card per manifest command; selecting a card builds a form whose inputs come from a single table — argument type to widget. An enum argument becomes a select box with the declared options; ipv4, mac and url become text inputs with type-appropriate placeholders; json becomes a textarea with live parsing; int and float become number inputs; bool becomes a two-state select. Adding a command to the manifest makes it appear in the UI with a working form, and no frontend change ships with it.

Arg type	Widget	Notes
enum	select	options come from the manifest, never hardcoded
ipv4 / mac / url / path	text input	placeholder from the arg's help text
int / float	number input	defaults pre-filled from the manifest

Arg type	Widget	Notes
bool	two-state select	explicit true / false, no silent defaults
json	textarea + parser	invalid JSON stays a string; the runner judges
str	text input	help text as placeholder

Table 5 — the widget table: the whole generative-UI trick in six rows.

Danger levels drive the confirm flow: a *destructive* command opens a confirmation dialog, and the engine independently refuses it unless the confirmation flag was passed — two gates, UI and engine, because a bench that can silently power off fifty screens is a liability. Outcomes render as structured JSON with emitted events called out, and every run books a coverage cell, so the console itself is a testing instrument. The remaining views are equally derived: events as a table of source and payload type, servers as cards with port and health route, limits as a table naming the component each one governs.

9. The coverage map

Coverage here is behavioural, not line coverage. The ledger records every invocation under a cell key of workspace, command and **argument bucket** — the bucket is derived from the invocation's scalar arguments, so vendor=roku and vendor=samsung are different cells of the same command. The heatmap is then simply the manifest surface minus the filled cells, with four statuses: *pass* (at least one passing cell), *partial* (refused-only — valuable, it proves the refusal vocabulary holds there), *failed* (runs but never passed), and *uncovered* (never touched).

The ledger refuses to record lab-internal failures as cell outcomes: a broken manifest says nothing about the workspace under test and would poison the map. Reports come out as markdown for CI artifacts and PR comments, and JSON for the shell. The same file that backs the UI heatmap can therefore gate a merge: cover the commands a release touches, or the release does not ship.

```
## Lab coverage — tv-rs (v0.6.0)

| Command | Label | Danger | Buckets | Runs | Pass | Refused | Failed | Status |
|---|---|---|---:|---:|---:|---:|---:|---|
| `tv.power.wake` | Wake (WoL) | safe | 1 | 1 | 1 | 0 | 0 | pass |
| `tv.power.off` | Power off | destructive | 0 | 0 | 0 | 0 | 0 | uncovered |
| `tv.panel.orientation` | Panel orientation (LFD) | attention | 0 | 0 | 0 | 0 | 0 | uncovered |

**Coverage: 40.0%**
```

Real output of `lab run scenarios/tv-morning-loop.toml` after one pass of the morning playlist.

10. The scenario composer: cross-crate pipelines

Because both workspaces speak the same vocabulary, a playlist can chain across them in one timeline: discover what answers on the LAN, wake the screens, verify power, launch content on the signage displays, pull a snapshot from the camera watching the entrance, and reconcile both fleets to desired state. The composer view in the shell is a plain editor over playlist TOML with a run button and the timeline renderer beside it — deliberately unfancy, because the interesting artifact is the playlist, and playlists belong in version control next to the manifests they exercise.

The runner shares one ledger across workspaces, so a cross-crate playlist fills two heatmaps at once and the report interleaves tv and camera steps with their outcomes and emitted events. A media push to a DLNA renderer can feed $vars.url forward from a Rhai step that asked tv-rs-media for a playlist URL; the same URL later appears in a camera timeline step as the thing the monitored screen was showing. That is the composer's real job: making the three workspaces *one story* about a building instead of three disconnected test suites.

11. Limit fuzzing and deterministic chaos

Limits are declared so they can be attacked on purpose. The rate limiter is a real token bucket per command, seeded from the manifest, and refusals arrive as LabError::RateLimited with a computed retry delay — the UI can literally show *retry after 2500 ms* instead of a generic failure. Chaos injection is counter-based: fail_every_nth, poison_after and extra_latency_ms, no RNG anywhere, which is what makes a failing run reproducible. A scenario that fails on call thirty-one will fail on call thirty-one tomorrow, on any machine, until the policy changes.

The fuzz targets fall out of the manifest's limits table: the budgeted WebSocket frame reader in tv-rs-transport and the budgeted FrameReader in cctv-transport get flooded with slow-drip and oversized frames; the per-TV connection pool gets more concurrent pairs than sockets; the agent's remediation ladder gets a device that flaps and one that never pairs, mirroring tv-rs's own Phase 4 exit test — fifty devices, twenty-four simulated hours, three blips, one device whose power is never observable, and the invariants that every healthy device converges hourly and no device exceeds one action per hour. The lab's contribution is that those scenarios run from the same playlists, in the same UI, with the same ledger, instead of living as one-off test binaries.

1 action / hour
per-device reconciler invariant under fuzz

fail_every_nth
deterministic chaos: no RNG, replayable

50 devices
Phase-4 style simulated fleet scenario

12. Expansion: the generative roadmap

The manifest vocabulary was designed to stretch, and the expansion ideas are ordered by how little new concepts they need. Each row below is a concrete direction with the piece of the lab it reuses and the one new thing it brings.

Idea	What it reuses	The new piece
mediamaster-rs manifests	commands/events vocabulary, runner, ledger	audio zones and sensors as device classes; EventSource::bus is already declared
Android TV (ATR v2) backend	capability sets, mock-first runner	one more vendor enum mapping; the pairing flow joins Auth
Hardware toggle	CommandSink port, whole scenario suite	a CommandSink over TvClient / cctv facade; pinning + quirks already in the workspaces
WASM scenario plugins	playlist runner contract	guest modules replace Rhai steps where polyglot logic matters
Snapshot diffing	coverage ledger cells	state snapshots per cell; diff two runs of the same playlist
AI-assisted scenario synthesis	manifest JSON as prompt context	an LLM proposes playlists; the runner + ledger keep it honest
Golden traces	step reports	record once on hardware, replay against mocks in CI
Release gate	drift view + coverage markdown	publish refuses when manifest behind facade or coverage regressed

Table 6 — the expansion roadmap, cheapest first.

Two of these deserve emphasis. **Hardware toggle** is deliberately a typed refusal today — calling set_backend returns a capability error that names exactly what is missing — because an honest 'not yet' beats a half-working adapter. **AI-assisted synthesis** is the most generative direction and the most dangerous if done naively; the lab makes it safe by construction, because the model can only speak the playlist and manifest languages, the runner enforces the limits, and the ledger measures whether the generated scenarios actually covered anything. The machine proposes; the same artifact discipline that merges generated manifests disposes.

13. Roadmap and getting started

The scaffold is buildable and tested today: seven crates, thirty-four unit tests, green on stable Rust with no network needed, plus two sample manifests, two sample playlists and a real generator binary. The shell's frontend and Rust commands are complete enough to run once the platform webview packages are present. The recommended order from here is small and concrete.

Phase	Deliverable	Exit criterion
0.1 — now	core crates + shell scaffold, mock fleet	cargo test green; shell builds with webview deps installed
0.2	hardware CommandSink over tv-rs facade	tv-morning-loop passes against one real Roku; refusals identical to mocks
0.3	cctv adapter + snapshot proxy server control	cctv-zone-watch passes on one ONVIF camera; StreamMux limits fuzzed
0.4	coverage gate + drift gate in CI	publish blocked on regression or stale manifest
0.5	mediamaster manifests + WASM plugin slot	one cross-workspace playlist drives all three workspaces

Table 7 — phases with exit criteria, in the spirit of the workspaces' own plan docs.

Getting started is three commands: cargo test --workspace proves the core; cargo run -p lab-engine --bin lab -- run scenarios/tv-morning-loop.toml --manifest manifests/tv-rs.lab.toml --confirm runs the first playlist headless and prints the coverage table; cd shell && npm install && npm run tauri dev opens the bench. From there, the fastest way to feel the design is to add one command to tv-rs.lab.toml, watch it appear in the UI with a generated form, run it against the mock fleet, and find it in the heatmap two clicks later.

The repository layout follows the same discipline as the workspaces it tests: crates by role, manifests as data, scenarios as data, the shell excluded from the core workspace so the GUI's system dependencies never reach the test profile. Nothing in the core pulls a GUI toolkit; nothing in the shell re-implements the engine; nothing in either invents a second error vocabulary.